

AI Fashion Personalization Platform

Production-Level Implementation, Architecture & Data Schema Plan

1. Product Overview

Vision

Build an AI-powered fashion personalization platform that:

- learns user aesthetic preferences
- understands style taste and fit preferences
- retrieves current fashion trends dynamically
- generates personalized outfit concepts
- allows interactive refinement after generation
- continuously improves personalization through user interaction

The platform is NOT a generic AI image generator.

It is:

A Fashion Taste Intelligence System

that combines:

- recommendation systems
 - vector similarity search
 - retrieval-augmented trend intelligence
 - controllable AI generation
 - user taste embeddings
 - conversational refinement
-

2. Core Product Goals

MVP Goals

The first production-ready version should:

1. onboard users through preference learning
2. create fashion taste embeddings
3. retrieve relevant fashion trends
4. generate AI outfit concepts

Signed

5. allow post-generation refinement
 6. store user fashion memory
 7. provide fast interaction on local-machine-capable infrastructure
-

3. Local Machine Constraints & Optimal Technology Choices

Important Constraint

The architecture must work efficiently on:

- mid-range laptop/desktop
- limited VRAM GPUs
- local development environment
- affordable deployment later

Recommended minimum:

Component	Recommended
RAM	16 GB
GPU	RTX 3050 / 3060 / 4050 / 4060
VRAM	6–8 GB
CPU	Ryzen 5 / Intel i5
Storage	SSD

The implementation plan is intentionally optimized for:

- modularity
 - incremental scaling
 - low hardware overhead initially
-

4. Recommended Tech Stack

Frontend

Layer	Technology
Framework	Next.js 15
Styling	Tailwind CSS
State Management	Zustand

Layer	Technology
UI Components	shadcn/ui
Animations	Framer Motion
API Layer	Axios
Authentication	Firebase Auth

Reason:

- modern ecosystem
- fast rendering
- SSR support
- excellent UI capability
- lightweight state management

Backend

Layer	Technology
API Framework	FastAPI
Async Server	Uvicorn
Validation	Pydantic
ORM	SQLAlchemy
Authentication	JWT
Background Tasks	Celery
Task Queue	Redis

Reason:

- FastAPI performs very well
- async support
- Python AI ecosystem integration
- excellent ML compatibility

AI/ML Stack

Component	Technology
Image Generation	Stable Diffusion XL
Fine-Tuning	LoRA

Component	Technology
Control Layer	ControlNet
Embeddings	CLIP
Recommendation Embeddings	SentenceTransformers
Image Similarity	FAISS
Prompt Orchestration	LangChain (light usage only)

Reason:

- SDXL can run locally
- LoRA reduces training requirements
- CLIP is ideal for fashion similarity
- FAISS is lightweight and fast

Databases

Purpose	Technology
Main relational DB	PostgreSQL
Vector search	FAISS
Cache/session	Redis
Object storage	Local initially / S3 later

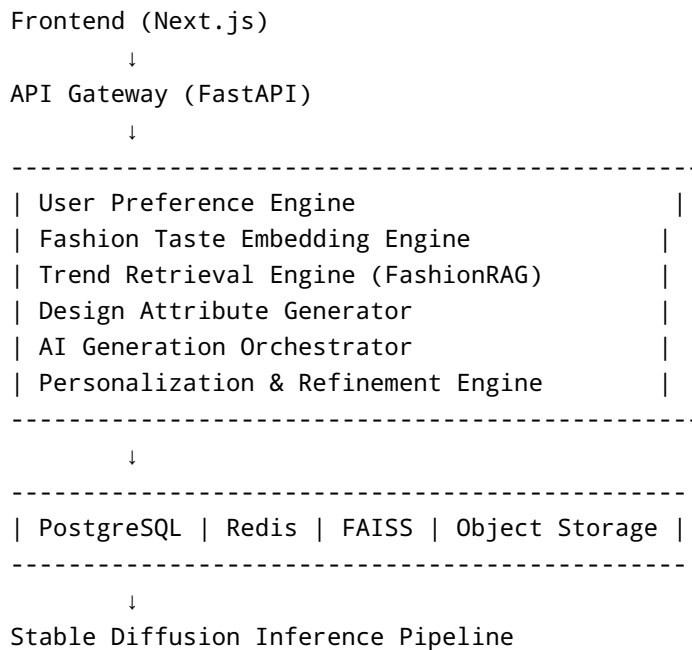
Reason:

- PostgreSQL is scalable and reliable
- FAISS is ideal locally
- Redis improves responsiveness

Deployment

Layer	Technology
Containerization	Docker
Reverse Proxy	Nginx
Frontend Hosting	Vercel
Backend Hosting	Render / Railway / VPS
GPU Inference Later	Modal / RunPod

5. High-Level System Architecture



6. Core System Modules

Module 1 — User Preference Engine

Purpose

Convert vague user preferences into structured aesthetic signals.

Inputs

Style Preferences

Examples:

- Korean
- Streetwear
- Casual
- Formal
- Vintage
- Indo-Western
- Minimalist
- Techwear

Interaction Inputs

- likes/dislikes
 - swipe actions
 - saved outfits
 - viewed outfits
 - refinement actions
 - prompt edits
-

Output

Structured taste profile.

Module 2 — Fashion Taste Embedding Engine

Purpose

Represent user fashion taste numerically.

This becomes the personalization core.

Embedding Pipeline

Inputs

- liked outfit images
 - disliked outfit images
 - text preferences
 - style selections
 - saved outfits
-

Embedding Model

Use:

- OpenCLIP ViT-B/32 initially

Reason:

- lightweight
 - excellent image-text similarity
 - local-machine friendly
-

Embedding Storage

Store vectors in:

- FAISS index

Metadata stored in PostgreSQL.

User Taste Vector Structure

Example:

```
{
  "user_id": "uuid",
  "style_weights": {
    "korean": 0.7,
    "casual": 0.2,
    "formal": 0.1
  },
  "fit_preference": "oversized",
  "color_preference": ["black", "beige"],
  "pattern_density": "low"
}
```

Module 3 — FashionRAG Trend Retrieval Engine

Purpose

Retrieve current fashion trends dynamically.

This prevents stale generation.

Initial Trend Sources

For MVP:

- curated local datasets
- manually tagged trend collections
- Pinterest dataset exports
- Kaggle fashion datasets

Do NOT scrape aggressively initially.

Trend Categories

Category	Examples
Color Trends	muted palettes
Fit Trends	oversized fits
Style Trends	Korean layering
Texture Trends	matte fabrics
Seasonal Trends	summer streetwear

Retrieval Pipeline

```
User Context
  ↓
Generate Search Query
  ↓
FAISS Similarity Search
  ↓
Retrieve Relevant Trends
  ↓
Inject Into Prompt Builder
```

Module 4 — Design Attribute Generator

Purpose

Convert:

- taste
- trends
- mood
- occasion
- fit preferences

into:

structured fashion attributes.

This module is CRITICAL.

FINAL ATTRIBUTE TAXONOMY

A. Style Attributes

Field	Type
primary_style	enum
secondary_style	enum
aesthetic_type	enum
era_style	enum

B. Fit Attributes

Field	Type
fit_type	enum
sleeve_fit	enum
clothing_length	enum
waist_style	enum
layering_level	integer

C. Color Attributes

Field	Type
primary_color	string
secondary_color	string
palette_type	enum
contrast_level	enum

D. Pattern Attributes

Field	Type
pattern_type	enum

Field	Type
pattern_density	enum
embroidery_level	enum
texture_style	enum

E. Fabric Attributes

Field	Type
material_type	enum
softness_level	integer
breathability_level	integer
flexibility_level	integer

F. Mood Attributes

Field	Type
emotional_style	enum
energy_level	enum
confidence_level	integer
luxury_level	integer

G. Occasion Attributes

Field	Type
occasion_type	enum
season_type	enum
formality_level	enum
weather_type	enum

H. Cultural Fusion Attributes

Field	Type
korean_influence	integer
western_influence	integer
indian_fusion_level	integer
japanese_influence	integer

Module 5 — AI Generation Engine

Purpose

Generate controllable outfit concepts.

Recommended Model

SDXL Base + LoRA

Reason:

- local-machine friendly
 - high-quality outputs
 - easier customization
 - manageable VRAM requirements
-

Generation Pipeline

```
Structured Attributes
  ↓
Prompt Builder
  ↓
Trend Context Injection
  ↓
Generation Constraints
  ↓
SDXL Inference
  ↓
Generated Outfit Concept
```

Prompt Builder Structure

The system should NEVER directly use raw user text.

Instead:

Attributes → Controlled Prompt

Example Prompt

Korean oversized casual fashion,
minimal monochrome palette,
soft cotton texture,
muted black and beige colors,
layered streetwear aesthetic,
summer outfit,
elegant calm vibe,
modern youth fashion,
high-detail fashion design concept

Module 6 — Personalization & Refinement Engine

Purpose

Allow iterative design modification.

This is a key differentiator.

Refinement Types

Refinement	Example
Fit Adjustment	more oversized
Color Shift	darker palette
Style Fusion	more Korean
Pattern Editing	reduce embroidery

Refinement	Example
Texture Upgrade	premium fabric look
Mood Shift	more elegant

Important Architecture Rule

Refinement should be:

incremental editing

NOT:

complete regeneration

Recommended Technique

Use:

- latent-space editing
- prompt delta updates
- ControlNet guidance

Module 7 — User Fashion Memory System

Purpose

Persist evolving fashion taste.

Stored Information

Data	Usage
liked outfits	taste modeling
generation history	recommendations
saved designs	retrieval
refinement history	personalization

Data	Usage
session interactions	learning

7. Production Database Schema

USERS TABLE

```
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email VARCHAR(255) UNIQUE,
  username VARCHAR(100),
  profile_image TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

USER_STYLE_PROFILE TABLE

```
CREATE TABLE user_style_profiles (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  primary_style VARCHAR(50),
  secondary_style VARCHAR(50),
  fit_preference VARCHAR(50),
  palette_preference VARCHAR(50),
  mood_preference VARCHAR(50),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

USER_TASTE_EMBEDDINGS TABLE

```
CREATE TABLE user_taste_embeddings (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  embedding_path TEXT,
  embedding_version VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW()
);
```

FAISS stores actual vectors.

OUTFIT_GENERATIONS TABLE

```
CREATE TABLE outfit_generations (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  prompt TEXT,  
  image_url TEXT,  
  style_type VARCHAR(50),  
  mood_type VARCHAR(50),  
  fit_type VARCHAR(50),  
  generation_status VARCHAR(50),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

OUTFIT_REFINEMENTS TABLE

```
CREATE TABLE outfit_refinements (  
  id UUID PRIMARY KEY,  
  generation_id UUID REFERENCES outfit_generations(id),  
  refinement_prompt TEXT,  
  updated_image_url TEXT,  
  refinement_type VARCHAR(50),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

USER_INTERACTIONS TABLE

```
CREATE TABLE user_interactions (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  outfit_id UUID,  
  interaction_type VARCHAR(50),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

TREND_DATA TABLE

```
CREATE TABLE trend_data (  
  id UUID PRIMARY KEY,  
  trend_category VARCHAR(50),  
  trend_description TEXT,  
  embedding_reference TEXT,  
  season_type VARCHAR(50),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

8. FAISS Vector Architecture

Separate Indices

Use separate FAISS indices.

User Taste Index

Stores:

- user taste vectors

Outfit Similarity Index

Stores:

- generated outfit embeddings
- inspiration embeddings

Trend Retrieval Index

Stores:

- trend embeddings
 - style references
-

Recommended FAISS Type

Use:

IndexFlatIP

initially.

Reason:

- simpler
- accurate
- manageable scale

Later move to:

IVF + PQ

for scaling.

9. API Architecture

Auth APIs

Endpoint	Purpose
POST /auth/signup	registration
POST /auth/login	login
POST /auth/logout	logout

User Preference APIs

Endpoint	Purpose
POST /preferences/styles	save styles
POST /preferences/swipe	save interactions
GET /preferences/profile	fetch profile

Generation APIs

Endpoint	Purpose
POST /generate/outfit	generate design
POST /generate/refine	refine design
GET /generate/history	generation history

Trend APIs

Endpoint	Purpose
GET /trends/current	retrieve trends
GET /trends/recommended	personalized trends

Recommendation APIs

Endpoint	Purpose
GET /recommendations/outfits	similar outfits
GET /recommendations/styles	style recommendations

10. Frontend Architecture

Pages

Page	Purpose
Landing	product intro
Onboarding	style learning
Swipe Discovery	preference learning
Generator Dashboard	outfit generation
Refinement Studio	interactive editing
Saved Designs	history
Profile	personalization settings

Frontend State Structure

Use Zustand.

Recommended stores:

Store	Purpose
authStore	authentication
preferenceStore	taste preferences
generationStore	generation state
refinementStore	editing state
trendStore	trend retrieval

11. AI Inference Optimization

VERY Important For Local Machine

Do NOT run huge pipelines initially.

Recommended Optimizations

Use fp16 inference

Reduces VRAM.

Use xFormers attention

Improves memory efficiency.

Limit image resolution initially

Use:

768x768

for MVP.

Use queued generation

Do NOT allow unrestricted parallel generation.

Use Redis queue.

Cache embeddings aggressively

Avoid recomputing vectors.

12. Security Considerations

Authentication

Use:

- JWT access tokens
 - refresh tokens
 - Firebase authentication initially
-

Rate Limiting

Protect generation endpoints.

Use:

- slowapi
 - Redis-based throttling
-

Image Storage

Initially:

- local storage

Later:

- AWS S3
 - Cloudflare R2
-

13. Scalability Roadmap

Phase 1 — MVP

Single-machine architecture.

Phase 2 — Beta

Separate:

- frontend
 - backend
 - inference worker
-

Phase 3 — Scale

Introduce:

- Kubernetes
 - distributed inference
 - cloud GPU workers
 - CDN caching
-

14. Recommended Development Roadmap

Stage 1 — Core Backend

Build:

- FastAPI backend
 - PostgreSQL schema
 - JWT auth
 - Redis queue
-

Stage 2 — Preference System

Build:

- style onboarding

- swipe learning
 - taste profile storage
-

Stage 3 — Embedding Engine

Build:

- CLIP embedding pipeline
 - FAISS similarity search
 - recommendation retrieval
-

Stage 4 — Generation Pipeline

Build:

- SDXL generation
 - prompt builder
 - trend injection
-

Stage 5 — Refinement Studio

Build:

- iterative editing
 - refinement prompts
 - modular customization
-

Stage 6 — Production Hardening

Add:

- Docker
 - logging
 - monitoring
 - async workers
 - caching
 - optimization
-

15. Recommended Repository Structure

```
fashion-ai-platform/  
├── frontend/  
│   ├── app/  
│   ├── components/  
│   ├── store/  
│   ├── lib/  
│   └── styles/  
├── backend/  
│   ├── api/  
│   ├── models/  
│   ├── services/  
│   ├── embeddings/  
│   ├── generation/  
│   ├── retrieval/  
│   ├── workers/  
│   └── utils/  
├── ai-models/  
│   ├── loras/  
│   ├── checkpoints/  
│   └── controlnets/  
├── vector-db/  
│   └── faiss_indexes/  
├── datasets/  
├── docker/  
└── docs/
```

16. Critical Engineering Decisions

DO NOT

- generate directly from raw prompts
- overbuild cloud infrastructure initially
- use extremely large models early
- implement AR first
- attempt production tailoring patterns initially

MUST DO

- structured attributes
 - taste embeddings
 - incremental refinement
 - vector-based recommendation
 - trend retrieval
 - modular architecture
-

17. Final Product Definition

Finalized System

An AI-powered fashion personalization platform that learns user aesthetic taste, retrieves trend-aware fashion intelligence, generates controllable outfit concepts, and supports interactive collaborative refinement.

18. Final Engineering Recommendation

Your strongest engineering advantage is NOT image generation.

It is:

Fashion Taste Intelligence

Focus engineering effort primarily on:

- preference learning
- embeddings
- retrieval
- controllable generation
- refinement architecture

because those create defensibility and product uniqueness.